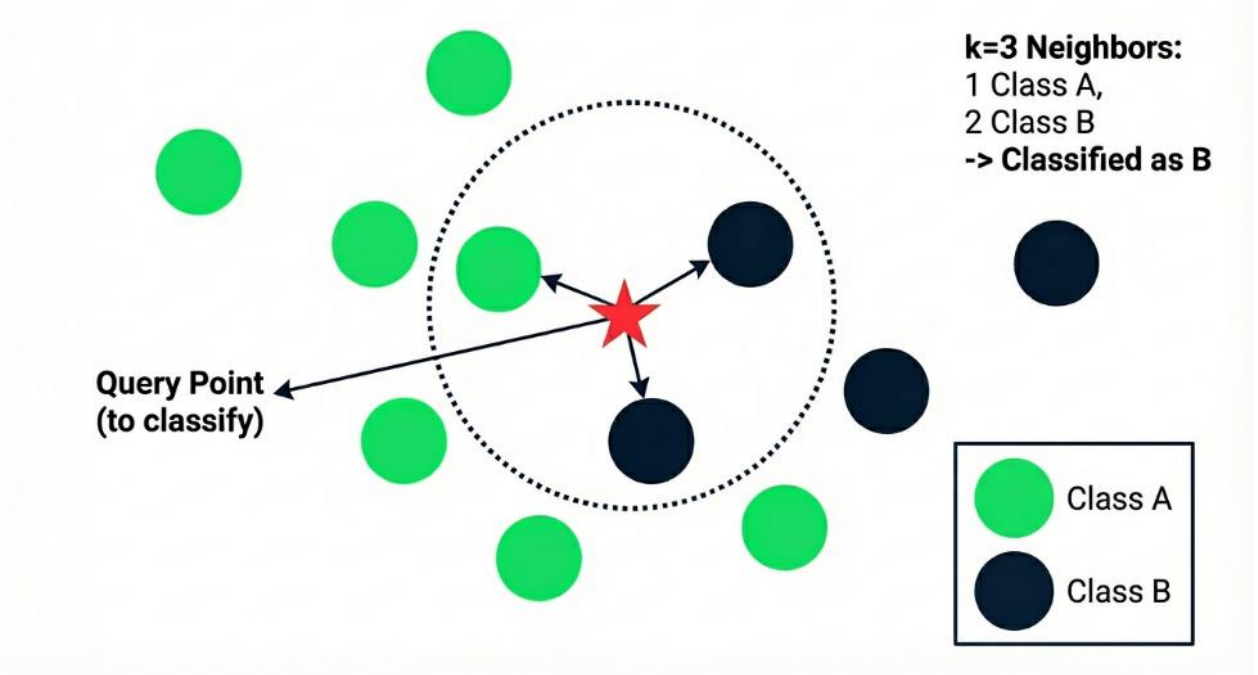
K-Nearest Neighbour (*k*NN) Classifier



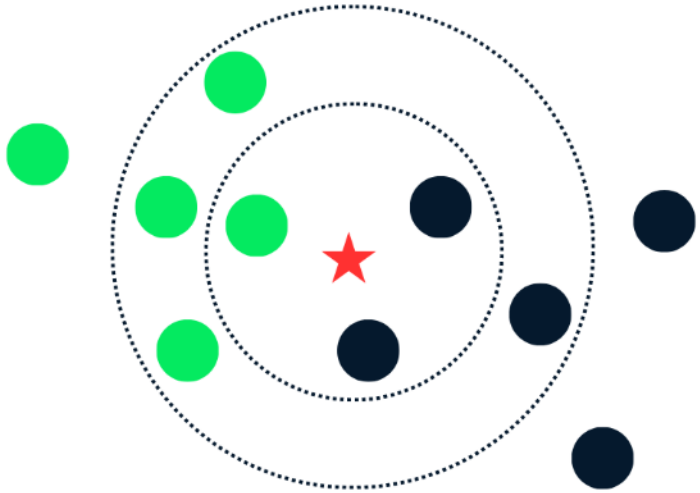
Dr. Abdelkarim Erradi
CSE@QU

With assistance from AI tools, particularly Microsoft Copilot and Google Gemini

Outline

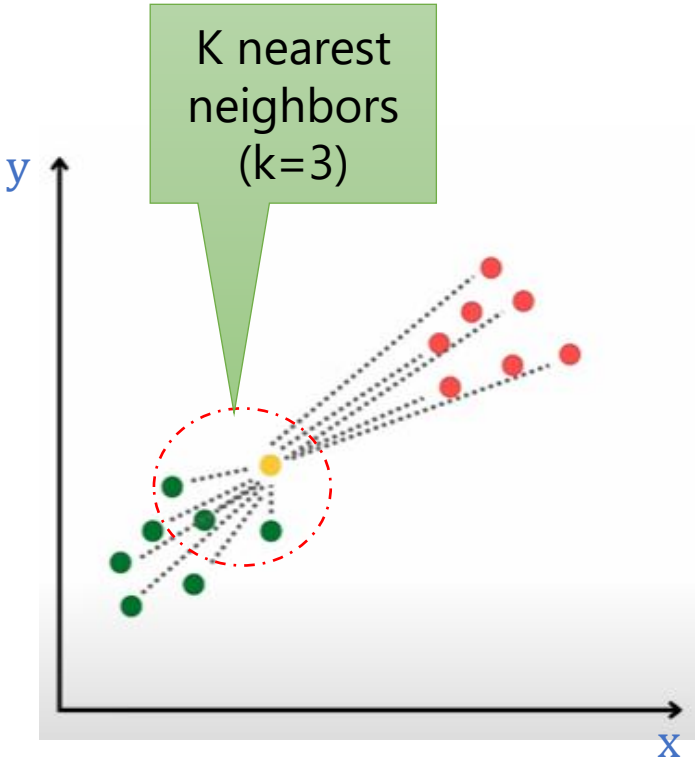
- K-Nearest Neighbour (kNN) Classifier
- Distance Metrics
- Model Hyperparameter Tuning

K-Nearest Neighbour (*k*NN) Classifier



kNN Algorithm steps

- kNN predicts the label of a new instance based on the labels of the most **similar** training examples.
- Given a data point that we want to classify:
  1. **Compute the distance** between the new instance and all training examples
  2. **Locate the k nearest neighbors** with the smallest distances
  3. For classification: Assign the **majority class** among the k neighbors
  3. For regression: Compute the **average value** of the k neighbors



kNN Key Characteristics (1 of 2)

- 🐌 **Lazy (Instance-Based) Learner**:
  - It does not build an **explicit model** during training
  - All computation happens **at prediction time** using stored training instances

  👉 Ideal when data changes frequently because no retraining is required
- 📊 **Non-parametric** Algorithm:

Makes no assumptions about the underlying data distribution

- 🔍 **Similarity-based** learning:
  - Assumes that similar instances have similar labels
  - Classifies new examples based on their closest neighbors in feature space

kNN Key Characteristics (2 of 2)

- 🧩 **Simple to Implement**
  Easy to understand and apply; often a strong baseline method.

- 📏 **Requires Feature Scaling**
  Distance can be dominated by features with large numeric ranges (e.g., height 1.5–1.8 m, weight 90–300 lb, income 10k–500k QR)

- ⚙️ **Minimal Tuning**
  Only **k** and the **distance metric**

- 👍 **Surprisingly Effective**
  Often performs well—worth trying first on many problems.

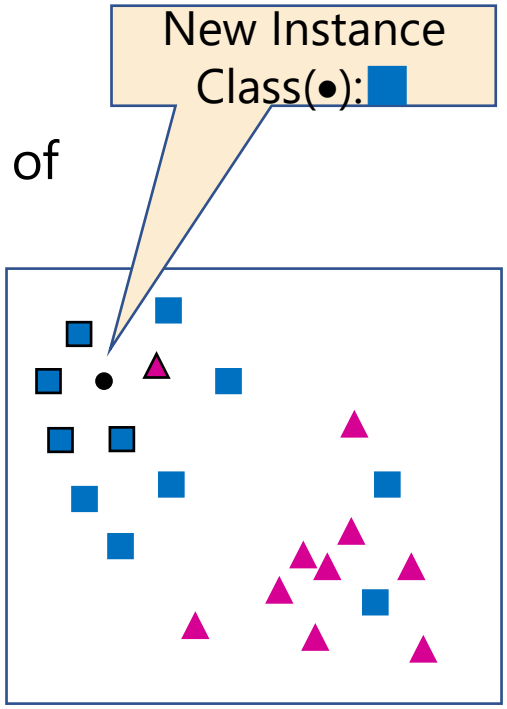
- ⏳ **Expensive at Prediction Time**
  Classifying a new instance requires scanning the **entire dataset**, which becomes 🐢 slow for large datasets.

- 📉 **Performs poorly in high dimensions** since accuracy degrades as the number of features grows.

- 📏 **Sensitive to the distance metric**; different metrics can produce different results

Requirements & Steps

- **Requirements:**
  - The set of training instances
  - Distance Metric to compute distance between instances
  - The value of the hyperparameter *k*, i.e., the number of nearest neighbors to consider
- **To predict an unknown instance:**
  - Compute distances to all training instances
  - Locate ***k*** nearest neighbors
  - **Classify using majority vote** among the k neighbors
  - In the case of a tie:
    - Use odd *k*
    - Choose the class with the **closest neighbor**
    - Pick the class dominant in the entire dataset
    - Decrease *k* by 1 until you break the tie



kNN in Python with scikit-learn

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline

# 1. Load data: 150 flowers, 4 measurements, 3 species
iris = load_iris()
X, y = iris.data, iris.target

# 2. Hold out 20% of the data for testing
# stratify=y ensures that the training and test sets maintain approximately the same
three flower class proportions as the original dataset.
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# 3. Scale the features, then fit kNN with k = 5
model = make_pipeline(StandardScaler(),
                      KNeighborsClassifier(n_neighbors=5))
model.fit(X_train, y_train)

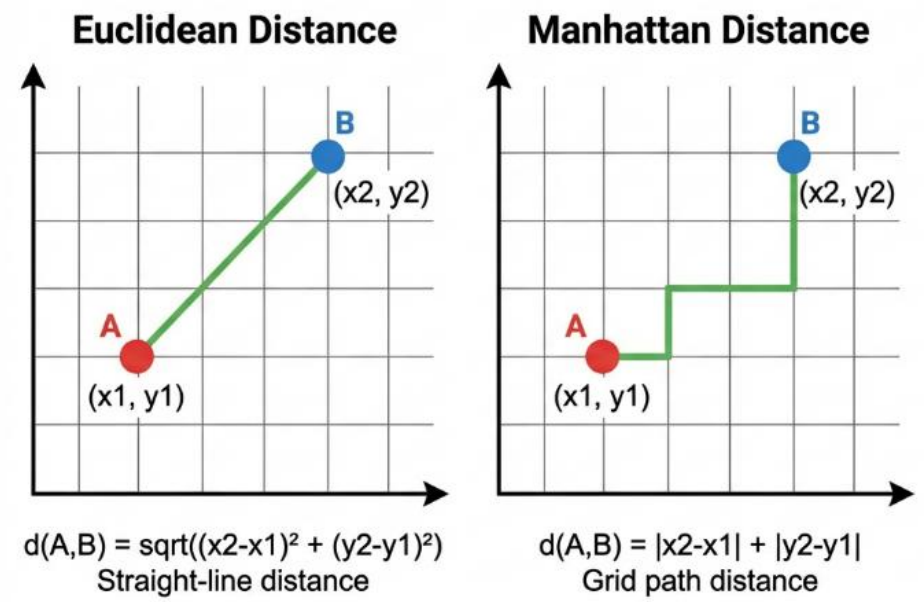
# 4. Evaluate on unseen data
print("Accuracy:", model.score(X_test, y_test))  # 0.933
```

Why scale?
kNN relies on distances, so large-scale features would dominate.

fit() = store
A lazy learner just memorizes the training data.

score() = accuracy
Share of test flowers classified correctly.

Distance Metrics



Popular Distance Metrics

Euclidean (L2)

$$d_{Eucl}(a, b) = \sqrt{\sum_{k=1}^{m} (a_k - b_k)^2}$$

- Straight-line distance
- Diagonal moves allowed: the shortest possible path

Example: A = (1, 2), B = (4, 6)

$$d = \sqrt{3^2 + 4^2} = 5$$

Best for: continuous features, low dimensions
Outliers: sensitive, squares large gaps

Manhattan (L1)

$$d_{Manh}(a, b) = \sum_{k=1}^{m} |a_k - b_k|$$

- Sum of absolute differences
- Moves only along axes, like city blocks: no diagonal shortcut

Example: A = (1, 2), B = (4, 6)

d = |4 − 1| + |6 − 2| = 3 + 4 = 7

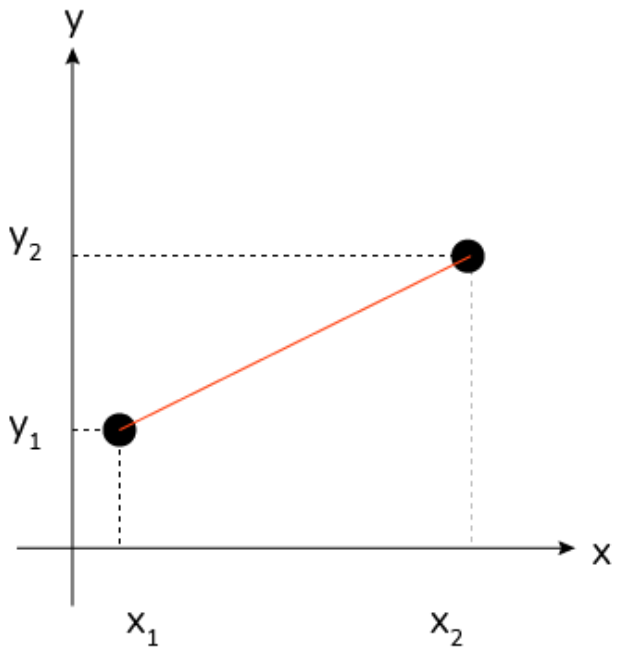
Walk 3 blocks east + 4 blocks north = 7 blocks

Best for: sparse, high-dimensional or noisy data
Outliers: more robust to noise

In kNN, choose it with **metric="euclidean"** or **metric="manhattan"**

Euclidean Distance

- **Euclidean Distance** is a measure of the **straight-line distance** between two points $(x_1, y_1)(x_2, y_2)$ in a two-dimensional space



It's derived from the **Pythagorean** theorem

$$\text{Distance} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- Example: A = (1, 2), B = (4, 6)
  - $d = \sqrt{(4 - 1)^2 + (6 - 2)^2} = \sqrt{25} = 5$
  - Smaller distance = more similar = nearer neighbour in kNN

Manhattan Distance

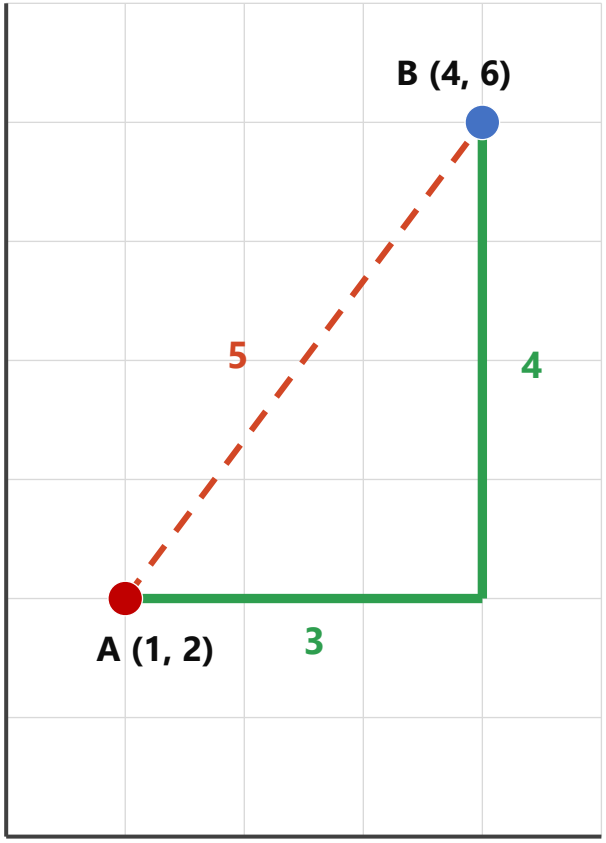
- Sum of the **absolute differences** between coordinates
- Like driving city blocks: horizontal and vertical moves only, no diagonal shortcuts

$$Distance = |x_2 - x_1| + |y_2 - y_1|$$

Example: A = (1, 2), B = (4, 6)

d = |4 − 1| + |6 − 2| = 3 + 4 = 7

Euclidean gave 5 for the same points



Manhattan: 3 + 4 = 7

Euclidean: 5 (dashed)

Use it when features are independent counts, sparse, or noisy: one large gap hurts less than with Euclidean.

Distance for Multiple Features

- When dealing with instances (data objects) with multiple $\mathbf{n}$ features, the Euclidean distance between two instances $\mathbf{p}$ and $\mathbf{q}$ with features ($p_1, p_2, \ldots p_n$) and ($q_1, q_2, \ldots q_n$) respectively, is given by:

$$\textbf{Euclidean} = \sqrt{\sum_{i=1}^{n} (q_i - p_i)^2}$$

- p_i and q_i represent the values of the ith feature for data objects $\mathbf{p}$ and $\mathbf{q}$ respectively
- $\mathbf{n}$ is the number of features

$$\textbf{Manhattan} = \sum_{i=1}^{n} |q_i - p_i|$$

Example: p = (2, 5, 1), q = (4, 1, 3)
Euclidean = √24 ≈ 4.90 Manhattan = 2 + 4 + 2 = 8



slides\knn.xlsx

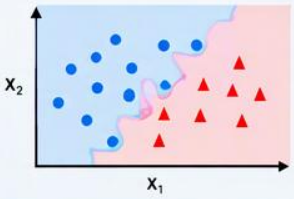
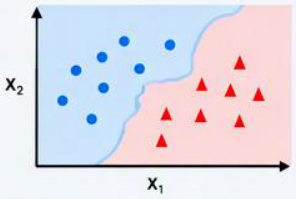
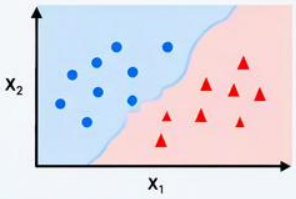
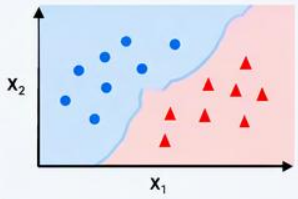
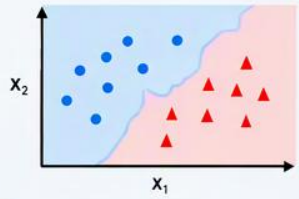
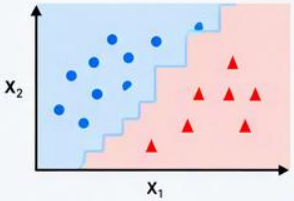
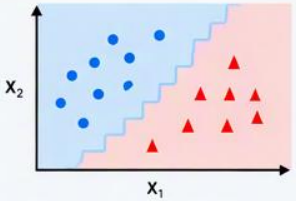
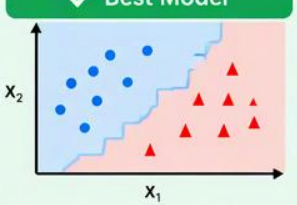
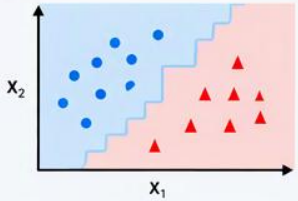
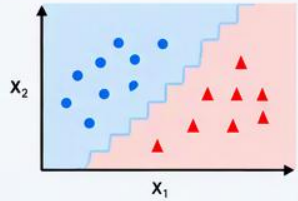
Model Hyperparameter Tuning

1. Train models
Use the same training data for all combinations of k and distance metric.

2. Try hyperparameter values
k ∈ {1, 3, 5, 7, 9}
distance metric ∈ {Euclidean, Manhattan}

3. Evaluate performance
Use a validation set (e.g., accuracy) for each combination.

4. Choose best model
Select the k and distance metric with the highest validation accuracy.

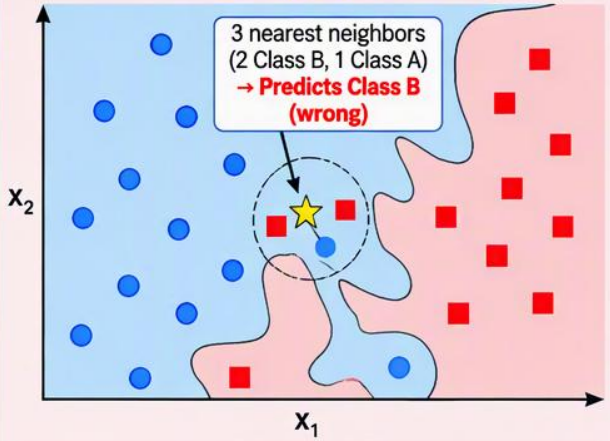
	k = 1	k = 3	k = 5	k = 7	k = 9
Euclidean distance	 Validation Accuracy **0.82**	 Validation Accuracy **0.85**	 Validation Accuracy **0.88**	 Validation Accuracy **0.86**	 Validation Accuracy **0.83**
Manhattan distance	 Validation Accuracy **0.80**	 Validation Accuracy **0.84**	✓ Best Model  Validation Accuracy **0.90**	 Validation Accuracy **0.87**	 Validation Accuracy **0.84**

Choosing k: Underfitting vs. Good Fit vs. Overfitting

Overfitting (k = 3)

Small k → very complex boundary (high variance)

Example: k = 3

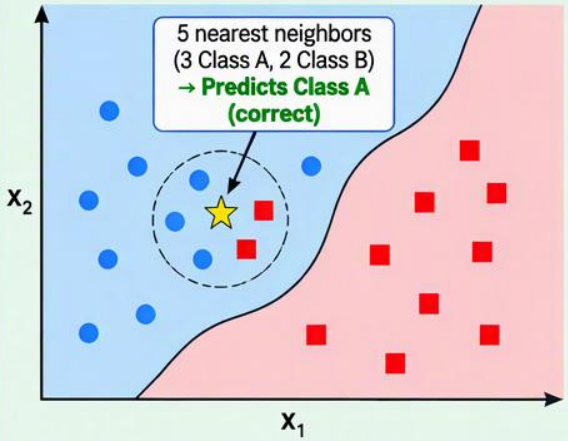


- It **memorizes** the training data, including noise (**poor generalization**).
- **High variance** = small data changes ⇒ **big prediction changes.**
- Here, the query point (☆) is **misclassified** as Class B because its 3 nearest neighbors include 2 B-class points, even though it lies in the A region.

Good Fit (k = 5)

Moderate k → good balance

Example: k = 5

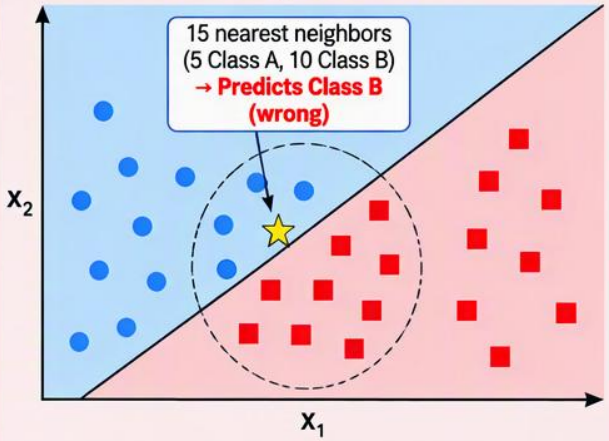


- Captures the main pattern in the data.
- Less sensitive to noise.
- Good generalization performance.
- The query point (☆) is correctly classified as Class A.

Underfitting (k = 15)

Large k → too simple boundary (high bias)

Example: k = 15



- It **oversimplifies** by letting many neighbors of mixed classes vote.
- **High bias** = distant points **dominate the vote,** hiding local patterns.
- The query point (☆) is **misclassified** as Class B.

Choose an appropriate value of k using cross-validation to achieve the best generalization performance on unseen data.

Choosing the Distance Metric

```
# Straight-line distance (default)
KNeighborsClassifier(n_neighbors=5, metric="euclidean")

# Sum of absolute differences
KNeighborsClassifier(n_neighbors=5, metric="manhattan")
```

metric="euclidean"
Straight-line distance. scikit-learn's default.

metric="manhattan"
Sum of absolute differences. Worth trying with many features or outliers.

Pick by validation
Scale features first, then keep the metric with the best cross-validated score.

No metric always wins: let's tune the model hyperparameters (next slide).

Tuning = Try different hyperparameter values, evaluate each using cross-validation, and select the best combination.

Tuning k and Metric with GridSearchCV

```
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline

# 1. Pipeline: scale features, then kNN
pipe = Pipeline([("scale", StandardScaler()),
           ("knn", KNeighborsClassifier())])

# 2. Candidate values to try
param_grid = {
    "knn__n_neighbors": list(range(1, 16, 2)),
    "knn__metric": ["euclidean", "manhattan"]}

# 3. 5-fold CV for every combination
grid = GridSearchCV(pipe, param_grid, cv=5)
grid.fit(X_train, y_train)
```

How it works

Grid: 8 values of k × 2 metrics = 16 combinations

Cost: 16 × 5 folds = 80 model fits

Safe: scaling sits inside the pipeline, so every fold is scaled on its own training part

Example: Iris flowers, 120 training samples; the test set stays untouched

Best Model and Final Test

```
# Best combination found by CV
print(grid.best_params_)
print(grid.best_score_)

# One final check on unseen data
best = grid.best_estimator_
print(best.score(X_test, y_test))
```

Output

Best: metric = euclidean, k = 5

CV accuracy: 0.967

Test accuracy: 0.933

Scale inside the pipeline

Stops test-fold data leaking into the scaler.

Keep the grid small

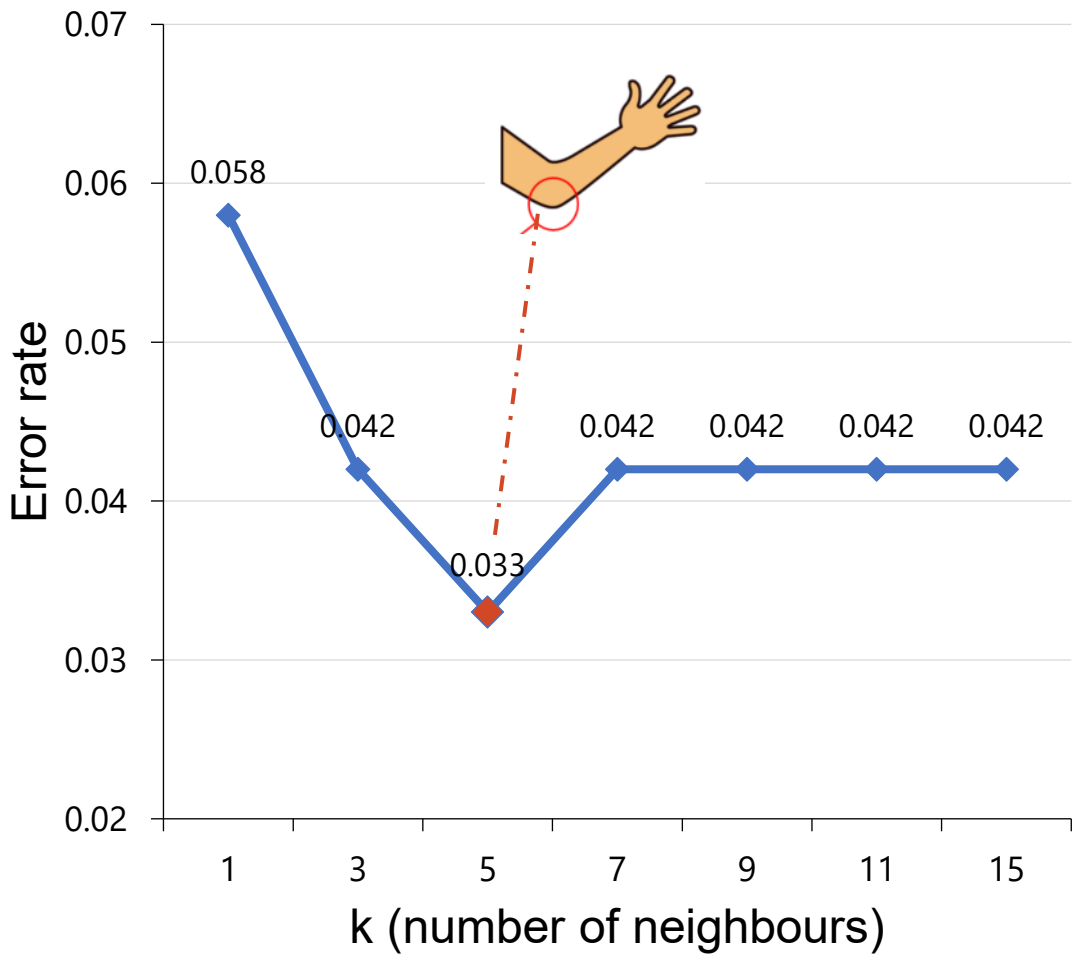
Fits = combinations × folds, so cost grows fast.

Flat results?

If scores tie, pick the smaller k: simpler and faster.

Tune on training data with cross-validation; use the test set for evaluation at the end.

Reading the Results: Error vs. k



What the curve shows

Lowest point: k = 5 with Euclidean (error 0.033)

Elbow: after it, a larger k gives no real gain

This elbow indicates the **best trade-off** between overfitting (small k) and underfitting (large k).

Metric: Manhattan is slightly worse here

Error rate = 1 − mean CV accuracy, taken from grid.cv_results_

Summary

- **How it works:** kNN classifies a new point by the majority vote of its k nearest training examples (average for regression).
- **Lazy & non-parametric:** no training phase and no data-distribution assumptions; all work happens at prediction time.
- **Distance matters:** Euclidean for continuous, low-dimensional data; Manhattan for sparse, noisy or high-dimensional data. Scale features first.
- **Choose k wisely:** small k overfits (high variance), large k underfits (high bias); pick the elbow of the error curve.
- **Tune with GridSearchCV:** search k and metric using cross-validation inside a pipeline, then evaluate on the test set.

Resources

- **User guide:** Nearest Neighbors in scikit-learn
  https://scikit-learn.org/stable/modules/neighbors.html
  https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html

- **Preprocessing:** Feature scaling (StandardScaler)
  https://scikit-learn.org/stable/modules/preprocessing.html

- **Tuning:** GridSearchCV and cross-validation
  https://scikit-learn.org/stable/modules/grid_search.html

- **Evaluation:** Confusion matrix, Precision, Recall, F1
  https://scikit-learn.org/stable/modules/model_evaluation.html